Big data analytics workflows with Databricks

Lecture 4 — Lakehouse & data modelling

Andrii Muzychuk
UCU, 2026

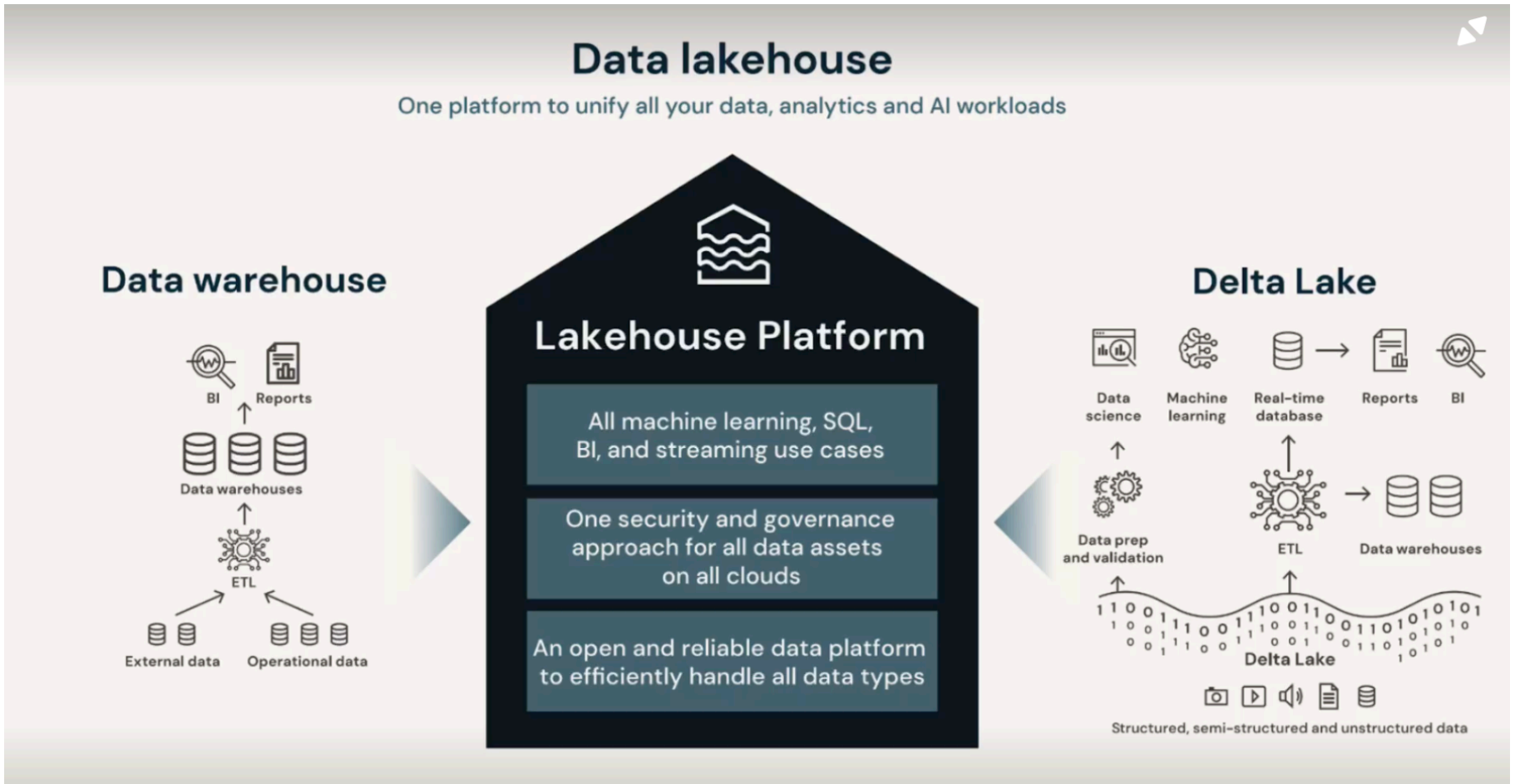
POWERED BY APPLIED SCIENCES FACULTY

Lecture 4 — Lakehouse & data modelling

Overview

- Lakehouse architecture
- Data modelling: Inmon, Kimball
- Data grain, 3NF, star & snowflake schema
- Slowly changing dimensions
- DWH architecture in a lakehouse: the medallion layers
- Dealing with bad/broken data

Lakehouse



The picture from lecture 1 — cheap flexible storage underneath, warehouse guarantees on top, one governance layer across everything.


Lakehouse
Streaming
Delta sharing
ETL
Logical Data Model
Data Model
Delta table
Slowly Changing Dimensions
Business Intelligence
Medallion architecture
Data Lake
Data Warehouse
Phisycal Data Model
Data Mart
Data Modelling
Kimball
Inman
Avro
Parquet
SCD
API
BI
AI
ML
SQL
DWH
ETL
Batch
Data Warehouse
Data Lake
Data Mart
Data Modelling
Phisycal Data Model
Business Intelligence
Slowly Changing Dimensions
Medallion architecture



Before we continue with the Lakehouse architecture, let's take a step back and refresh some of those keywords …

Data Modelling

> Structuring data that supports decision making

Data modelling steps

Business Model : Understand the business - actors, relationship, processes, requirements

Logical Data Model : a.k.a. UML diagram for business core concepts.

Physical Data Model : Physical data layout (e.g. DWH, tech specific)

Data Applications : Analytics, reporting, data apps, etc.

Logical data model

- **Entity**: person, place, concept
- **Attributes**: atomic piece of information describing an entity
- **Business rules**:
  - Key rules: identification of unique record
  - Domain rules: validation of attribute values

Data modelling schools

- Top-down, by Bill Inmon (1992)
- Bottom-up, by Ralph Kimball (1996)
- Data Vault 2.0, by Dan Linstedt (2015) (out of scope)

Data modelling schools

- Top-down, by Bill Inmon (1992)
- Bottom-up, by Ralph Kimball (1996)
- Data Vault 2.0, by Dan Linstedt (2015) (out of scope)

> **Tip**
>
> A more recent idea addresses the challenges of scalability and agility in centralized, monolithic data platforms like data warehouses and data lakes: **Data Mesh**, by Zhamak Dehghani (2019, out of scope)

Data modelling schools

- Top-down, by Bill Inmon (1992)
- Bottom-up, by Ralph Kimball (1996)
- Data Vault 2.0, by Dan Linstedt (2015) (out of scope)

> **Tip**
>
> A more recent idea addresses the challenges of scalability and agility in centralized, monolithic data platforms like data warehouses and data lakes: **Data Mesh**, by Zhamak Dehghani (2019, out of scope)

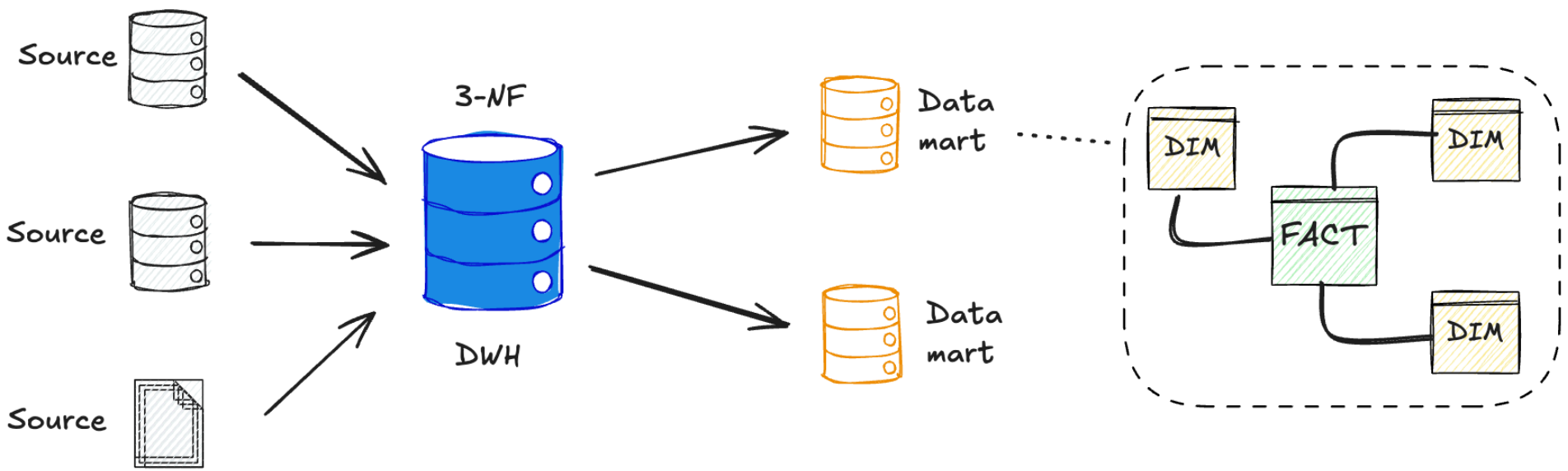
> **Tip**
>
> One big table model should not be ignored! Sometimes it can be more optimal choice.

Bill Inmon’s Top-down approach

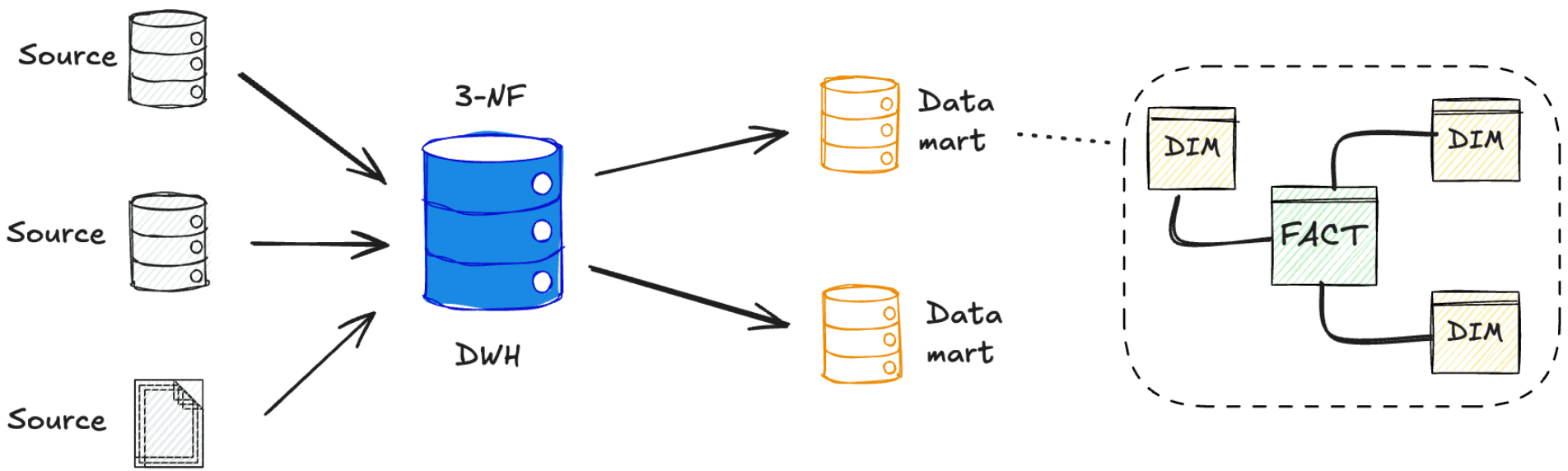
> Corporate Information Factory prioritizes building enterprise DWH that serves a single source of truth.

- Data categorized into a subjects rather than applications or processes
- DWH is built first which ensures data consistency, reduces redundancy
- Data marts are built later from a DWH to serve a specific business need
- DWH data is in 3NF (Normalization Form)

Bill Inmon’s Top-down approach

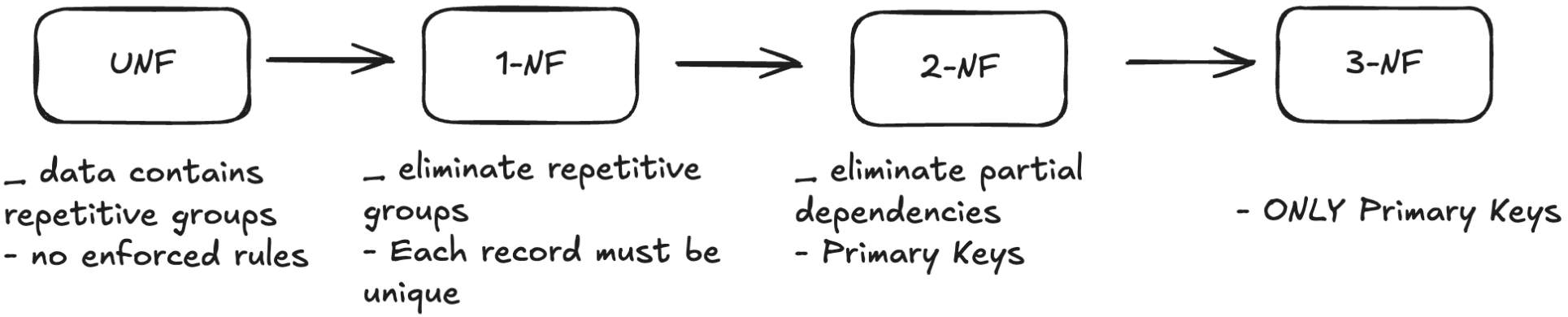


Bill Inmon's Top-down approach



- what exactly is 3NF?
- what is a data mart?

Data normalization forms



Note

- more normalization forms exists
- 3-NF is most commonly used in data warehousing

The example: a library

One table, as the librarian actually keeps it. **UNF — unnormalised.**

MemberID	Name	Address	Books borrowed
M100	Alice	123 Oak St, Anytown	101 The Hobbit / A1 Tolkien; 103 1984 / A2 Orwell; 104 Animal Farm / A2 Orwell
M101	Bob	456 Pine Ln, Otherville	102 Dune / A3 Herbert
M102	Carol	789 Maple Ave, Anytown	101 The Hobbit / A1 Tolkien

Tip

Alice has borrowed three books. All three live in **one cell** — and when she borrows a fourth, that cell just gets longer. What can you not do with this table?

Problem with UNF

- **You cannot query it.** “How many people borrowed The Hobbit?” means parsing a string.
- **You cannot key it.** There is no column, or combination of columns, that identifies one borrowing.
- **You cannot enforce anything.** No type, no foreign key, no constraint applies inside a text blob.

Problem with UNF

- **You cannot query it.** “How many people borrowed The Hobbit?” means parsing a string.
- **You cannot key it.** There is no column, or combination of columns, that identifies one borrowing.
- **You cannot enforce anything.** No type, no foreign key, no constraint applies inside a text blob.

> **Note**
>
> This is the shape data arrives in — a JSON payload, a spreadsheet, a log line. It is exactly what your **bronze** layer holds.

Problem with UNF

- **You cannot query it.** “How many people borrowed The Hobbit?” means parsing a string.
- **You cannot key it.** There is no column, or combination of columns, that identifies one borrowing.
- **You cannot enforce anything.** No type, no foreign key, no constraint applies inside a text blob.

> **Note**
>
> This is the shape data arrives in — a JSON payload, a spreadsheet, a log line. It is exactly what your **bronze** layer holds.

Fix: one value per cell, one row per thing that happened. → **1NF**

UNF → 1NF: one value per cell

Atomic values, and one row per *borrowing*. The key is now **(MemberID, BookID)** — composite, because neither column alone identifies a row.

MemberID	BookID	DueDate	Title	AuthorID	AuthorName	Name	Address
M100	101	2025-06-01	The Hobbit	A1	Tolkien	Alice	123 Oak St, Anytown
M100	103	2025-06-08	1984	A2	Orwell	Alice	123 Oak St, Anytown
M100	104	2025-06-12	Animal Farm	A2	Orwell	Alice	123 Oak St, Anytown
M101	102	2025-06-03	Dune	A3	Herbert	Bob	456 Pine Ln, Otherville
M102	101	2025-06-05	The Hobbit	A1	Tolkien	Carol	789 Maple Ave, Anytown

Tip

Now it is queryable. But look at what it cost.

What 1NF still does not fix

- **Redundancy**: Alice's address is stored **three times**, "The Hobbit / A1 / Tolkien" twice, and Orwell's name twice.
- **Update anomaly**: Alice moves. Her address is in two rows. Update one, miss the other, and the database now holds two different addresses for one person — with no way to say which is right.
- **Insert anomaly**: a new member who has borrowed nothing cannot be recorded at all — there is no BookID to complete the key.
- **Delete anomaly**: Bob returns Dune, you delete the row, and *Dune itself* disappears from the library.

What 1NF still does not fix

- **Redundancy**: Alice's address is stored **three times**, "The Hobbit / A1 / Tolkien" twice, and Orwell's name twice.
- **Update anomaly**: Alice moves. Her address is in two rows. Update one, miss the other, and the database now holds two different addresses for one person — with no way to say which is right.
- **Insert anomaly**: a new member who has borrowed nothing cannot be recorded at all — there is no BookID to complete the key.
- **Delete anomaly**: Bob returns Dune, you delete the row, and *Dune itself* disappears from the library.

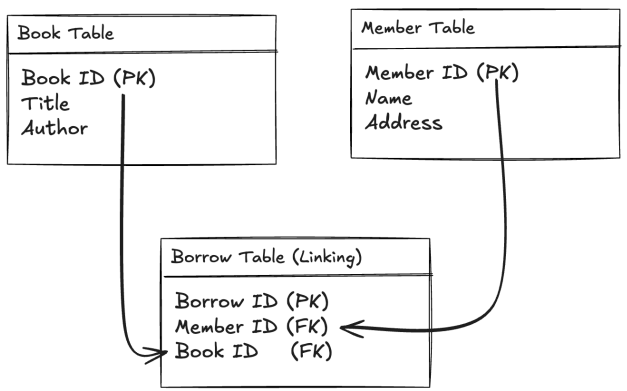
> **Important**
>
> Redundancy is a storage problem. The three anomalies are **correctness** problems, and they are the reason normalisation exists.

1NF → 2NF: remove partial dependencies

A **partial dependency** is a non-key column that depends on only *part* of the composite key.

- `Title`, `AuthorID`, `AuthorName` depend on **BookID** alone
- `Name`, `Address` depend on **MemberID** alone
- `DueDate` depends on **both** — it is the only column that describes the borrowing itself

2NF schema — each group split into its own table



Borrow

MemberID	BookID	DueDate
M100	101	2025-06-01
M100	103	2025-06-08
M100	104	2025-06-12
M101	102	2025-06-03
M102	101	2025-06-05

Book

BookID	Title	AuthorID	AuthorName
101	The Hobbit	A1	Tolkien
102	Dune	A3	Herbert
103	1984	A2	Orwell
104	Animal Farm	A2	Orwell

Member

MemberID	Name	Address
M100	Alice	123 Oak St, Anytown
M101	Bob	456 Pine Ln, Otherville
M102	Carol	789 Maple Ave, Anytown

Alice’s address is now stored **once**. All four anomalies are gone.

2NF → 3NF: remove transitive dependencies

A **transitive dependency** is a non-key column that depends on another *non-key* column.

Look at `Book`: `BookID → AuthorID → AuthorName`. The author's name does not depend on the book — it depends on the **author**.

2NF → 3NF: remove transitive dependencies

A **transitive dependency** is a non-key column that depends on another *non-key* column.

Look at `Book`: `BookID → AuthorID → AuthorName`. The author's name does not depend on the book — it depends on the **author**.

So the same anomaly is still hiding in there. **Orwell is stored twice** — once for *1984* and once for *Animal Farm*. Correct a misspelled author name and you must correct it in every one of their books.

2NF → 3NF: remove transitive dependencies

A **transitive dependency** is a non-key column that depends on another *non-key* column.

Look at `Book`: `BookID → AuthorID → AuthorName`. The author's name does not depend on the book — it depends on the **author**.

So the same anomaly is still hiding in there. **Orwell is stored twice** — once for *1984* and once for *Animal Farm*. Correct a misspelled author name and you must correct it in every one of their books.

Book

BookID	Title	AuthorID
101	The Hobbit	A1
102	Dune	A3
103	1984	A2
104	Animal Farm	A2

Author

AuthorID	AuthorName
A1	Tolkien
A2	Orwell
A3	Herbert

The four forms, on one example

	what it fixes	the library table
UNF	—	one row per member, borrows in a text blob
1NF	repeating groups	one row per borrowing; key is (MemberID, BookID)
2NF	partial dependencies	**Borrow** + **Book** + **Member**
3NF	transitive dependencies	…and **Author** split out of Book

The four forms, on one example

	what it fixes	the library table
UNF	—	one row per member, borrows in a text blob
1NF	repeating groups	one row per borrowing; key is (MemberID, BookID)
2NF	partial dependencies	**Borrow** + **Book** + **Member**
3NF	transitive dependencies	…and **Author** split out of Book

> **Note**
>
> Each step removes one class of duplicated fact. The end state — **every fact stored exactly once** — is what “normalised” means, and the price is joins.

From 3NF to facts and dimensions

Normalisation just did the fact/dimension split **for you**. Look at what came out:

3NF table	what it holds	becomes
`Borrow` (MemberID, BookID, DueDate)	an **event** — one row per thing that happened	**fact table**
`Member` (MemberID, Name, Address)	*who*	**dimension**
`Book` (BookID, Title, AuthorID)	*what*	**dimension**
`Author` (AuthorID, AuthorName)	more detail about the *what*	**snowflaked dimension**

From 3NF to facts and dimensions

Normalisation just did the fact/dimension split **for you**. Look at what came out:

3NF table	what it holds	becomes
`Borrow` (MemberID, BookID, DueDate)	an **event** — one row per thing that happened	**fact table**
`Member` (MemberID, Name, Address)	*who*	**dimension**
`Book` (BookID, Title, AuthorID)	*what*	**dimension**
`Author` (AuthorID, AuthorName)	more detail about the *what*	**snowflaked dimension**

> **Tip**
>
> The tell: the table whose key is **made of other tables' keys** is the fact. Tables that describe one thing are dimensions.

…and then Kimball goes backwards

Dimensional modelling takes that 3NF schema and **deliberately denormalises the dimensions** — folding `Author` back into `Book`.

…and then Kimball goes backwards

Dimensional modelling takes that 3NF schema and **deliberately denormalises the dimensions** — folding `Author` back into `Book`.

That is a step from 3NF *back* to 2NF, on purpose. It is safe because analytical tables are **not updated**: no updates, no update anomaly, so the reason for the rule does not apply. What you get back is one less join on every query.

…and then Kimball goes backwards

Dimensional modelling takes that 3NF schema and **deliberately denormalises the dimensions** — folding `Author` back into `Book`.

That is a step from 3NF *back* to 2NF, on purpose. It is safe because analytical tables are **not updated**: no updates, no update anomaly, so the reason for the rule does not apply. What you get back is one less join on every query.

> **Important**
>
> Which is the whole difference between the two shapes you are about to see:
>
> **Star schema** — dimensions denormalised.
> **Snowflake schema** — dimensions left in 3NF.

Dimensional Modelling Core

- **Fact tables** - measurable, granular events, such as sales, transactions, or clicks.
- **Dimension tables** - rich descriptive context, such as customer, product, time, and location.

> **Note**
>
> **Fact** table contains numbers and **dimension** table contains the meaning

Fact tables

> Central table that store measurable, quantitative data related to business process (e.g. transactions in the store)

- often is a time series
- include numeric metrics (e.g. sales amount, quantity)
- no intermediate aggregation
- no overwrite/update/delete operations
- append only operations
- contain foreign keys referencing dimension tables

Dimension tables

> Tables that provides descriptive attributes related to fact data (e.g. store address, store manager name).

- contains textual or categorical information (is a form of lookup data, e.g. product name, store address)
- is often denormalized to optimize query performance
- contain surrogate keys that helps to keep track of historical changes in data, e.g. customer moves to another place
- contain hierarchies that enables to drill down (e.g. from country to city)
- typically referred to as Slowly Changing Dimensions
- update frequency is way lower compared to fact table

Grain

> **Grain** is the answer to one question: *one row per what?*

Grain

> **Grain** is the answer to one question: *one row per what?*

It is the first thing to establish about any table, and the last thing you can change.

table	grain
`orders`	one row per **order**
`lineitem`	one row per **line of an order** (~4 per order)
`fct_orders`	one row per **order** — lines aggregated on the way in

Grain

> **Grain** is the answer to one question: *one row per what?*

It is the first thing to establish about any table, and the last thing you can change.

table	grain
`orders`	one row per **order**
`lineitem`	one row per **line of an order** (~4 per order)
`fct_orders`	one row per **order** — lines aggregated on the way in

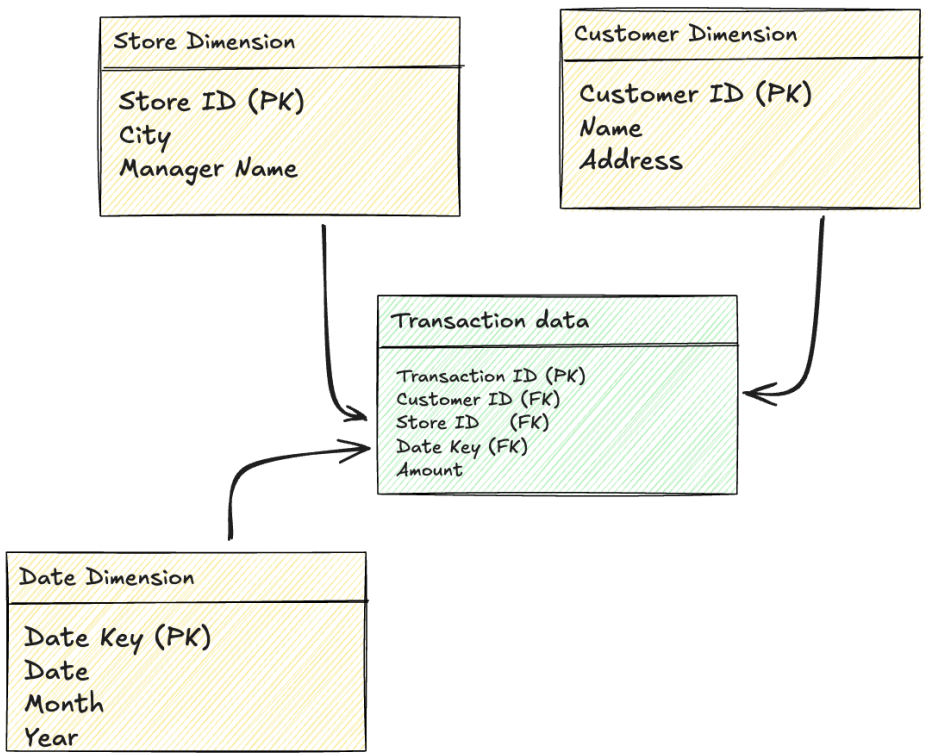
Important

Every measure on a fact table must be additive **at its grain**. And a layer that aggregates away the grain destroys every question that depended on the detail — which is why silver keeps it and gold is allowed to spend it.

Dimensional models

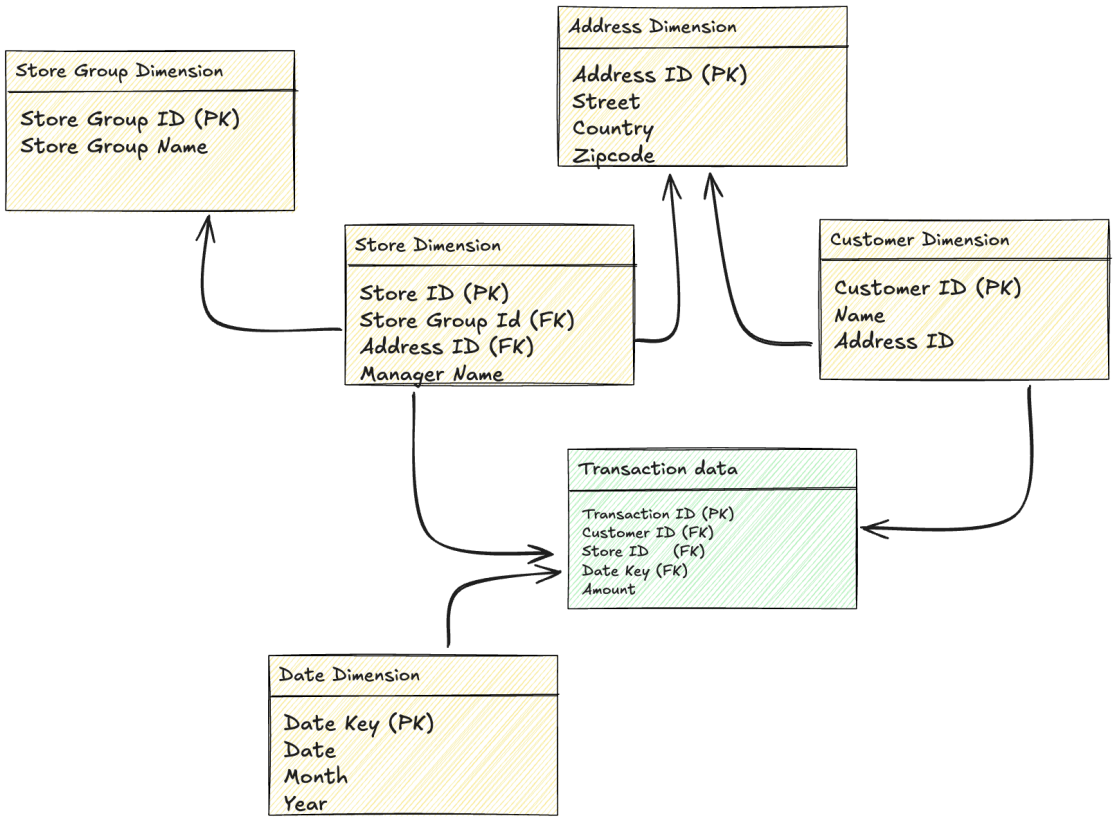
- Star schema
- Snowflake schema

Star Schema



- Fact table at the center connected with multiple dimension tables
- Improve readability and performance, simplifies analytical queries

Snowflake Schema



Note

Extension to star schema when dimension tables are further normalized into multiple related tables: reduces data redundancy, but may complicate analytical queries.

Types of dimensions

- **Conformed dimensions**: shared across multiple data marts ensuring consistency
- Role-playing dimensions: used multiple times within the same schema
- Junk dimension: combines unrelated low cardinality attributes to reduce clutter in fact table

Slowly changing dimensions

There are multiple strategies that we can implement to keep track historical changes in data and we refer to such dimension as Slowly changing dimensions.

- SCD Type 0 - No changes allowed (e.g. static lookup tables)
- SCD Type 1 - Overwrite old data with new data, do not preserve history
- SCD Type 2 - Creates a new record to preserve history

> **Note**
>
> There are other types, but SCD Type 1 and 2 are most widely used. Learn more: What is a slowly changing dimensions?

SCD Type 2 — two changes

Let imagine online service has two customers:

ID	Name	Address
M100	Alice	123 Oak St, Anytown
M101	Bob	456 Pine Ln, Otherville

> **Tip**
>
> Consider two scenarios: 1. Bob decided to change location on May 15, 2025. Think how to keep track this change? 2. Alice decides to unsubscribe on July 12, 2025. How to reflect change in this case?
>
> Think how to preserve this historical changes.

SCD Type 2 — the columns

Before we add new data …

sk	customer_id	name	address	is_active	is_current	effective_date	expiry_date
1	M100	Alice	123 Oak St, Anytown	true	true	2024-01-01	NULL
2	M101	Bob	456 Pine Ln, Otherville	true	true	2024-01-01	NULL



Tip

Think how new column can be useful and try to answer following questions:

- Why did we add the `sk` column?
- How does `is_current` help the business?
- What is the difference between `effective_date` and `expiry_date`?

SCD Type 2 — scenario 1: Bob moves

Bob decided to change location on May 15, 2025. How the table below will change?

sk	customer_id	name	address	is_active	is_current	effective_date	expiry_date
2	M101	Bob	456 Pine Ln, Otherville	true	true	2024-01-01	NULL

SCD Type 2 — scenario 1: Bob moves

Bob decided to change location on May 15, 2025. How the table below will change?

sk	customer_id	name	address	is_active	is_current	effective_date	expiry_date
2	M101	Bob	456 Pine Ln, Otherville	true	true	2024-01-01	NULL

sk	customer_id	name	address	is_active	is_current	effective_date	expiry_date
2	M101	Bob	456 Pine Ln, Otherville	true	false	2024-01-01	2025-05-15
3	M101	Bob	15 Forest Rd, Openair	true	true	2025-05-15	NULL

SCD Type 2 — scenario 2: Alice unsubscribes

Alice decides to unsubscribe on July 12, 2025. How the table below will change?

sk	customer_id	name	address	is_active	is_current	effective_date	expiry_date
1	M100	Alice	123 Oak St, Anytown	true	true	2024-01-01	NULL

SCD Type 2 — scenario 2: Alice unsubscribes

Alice decides to unsubscribe on July 12, 2025. How the table below will change?

sk	customer_id	name	address	is_active	is_current	effective_date	expiry_date
1	M100	Alice	123 Oak St, Anytown	true	true	2024-01-01	NULL

sk	customer_id	name	address	is_active	is_current	effective_date	expiry_date
1	M100	Alice	123 Oak St, Anytown	true	false	2024-01-01	2025-07-12
4	M100	Alice	123 Oak St, Anytown	false	true	2025-07-12	NULL

Why one MERGE is not enough

A change needs **two** things to happen: the current row is *closed*, and a new row is *opened*. A `MERGE` matches a row and either updates it or deletes it — never updates it *and* inserts a sibling for the same key.

Why one MERGE is not enough

A change needs **two** things to happen: the current row is *closed*, and a new row is *opened*. A MERGE matches a row and either updates it or deletes it — never updates it *and* inserts a sibling for the same key.

So: close first, insert second.

```sql
-- pass 1: close the rows that changed
MERGE INTO dim_customer t
USING customer_changes s
   ON t.customer_id = s.customer_id AND t.is_current = true
WHEN MATCHED THEN UPDATE SET
     t.is_current = false, t.expiry_date = s.change_date;
```

```sql
-- pass 2: open the new versions, with surrogate keys that cannot collide
INSERT INTO dim_customer
SELECT (SELECT max(sk) FROM dim_customer)
       + row_number() OVER (ORDER BY customer_id) AS sk,
       s.*, true AS is_current, s.change_date AS effective_date, NULL AS expiry_date
FROM customer_changes s;
```

The point-in-time query

This is what all those columns were for.

```sql
SELECT address, is_active
FROM   dim_customer
WHERE  customer_id = 42
  AND  effective_date <= DATE'1995-03-01'
  AND  (expiry_date IS NULL OR expiry_date > DATE'1995-03-01')
```

The point-in-time query

This is what all those columns were for.

```sql
SELECT address, is_active
FROM   dim_customer
WHERE  customer_id = 42
  AND  effective_date <= DATE'1995-03-01'
  AND  (expiry_date IS NULL OR expiry_date > DATE'1995-03-01')
```

Important

The interval is `[effective_date, expiry_date)` — effective inclusive, expiry **exclusive**, NULL for the current row. A closed row's `expiry_date` equals the next row's `effective_date`; that date belongs to the *later* version. Writing `>=` here is the most common SCD Type 2 bug.

The point-in-time query

This is what all those columns were for.

```sql
SELECT address, is_active
FROM   dim_customer
WHERE  customer_id = 42
  AND  effective_date <= DATE'1995-03-01'
  AND  (expiry_date IS NULL OR expiry_date > DATE'1995-03-01')
```

Important

The interval is `[effective_date, expiry_date)` — effective inclusive, expiry **exclusive**, NULL for the current row. A closed row's `expiry_date` equals the next row's `effective_date`; that date belongs to the *later* version. Writing `>=` here is the most common SCD Type 2 bug.

Against an SCD Type 1 dimension there is no as-of clause to write. The query cannot be expressed — 1995's orders are reported against today's address, and nobody notices.

Ralph Kimball's Bottom-up approach

> Prioritizes delivering value quickly by implementing one process at a time.

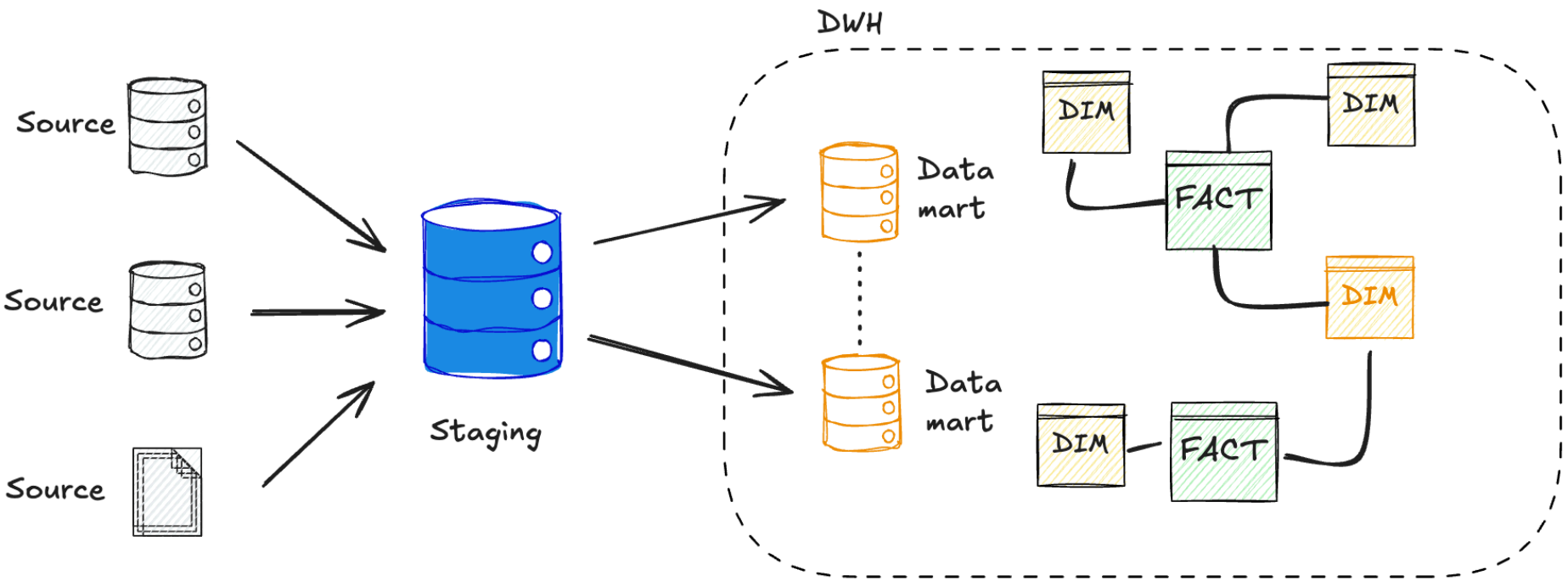
Key principles:

- Incremental development: build the DWH iteratively through data marts
- Dimensional design: organizes data into facts and dimension tables.
- Bus architecture: conformed dimensions - shared across DWH

Advantages:

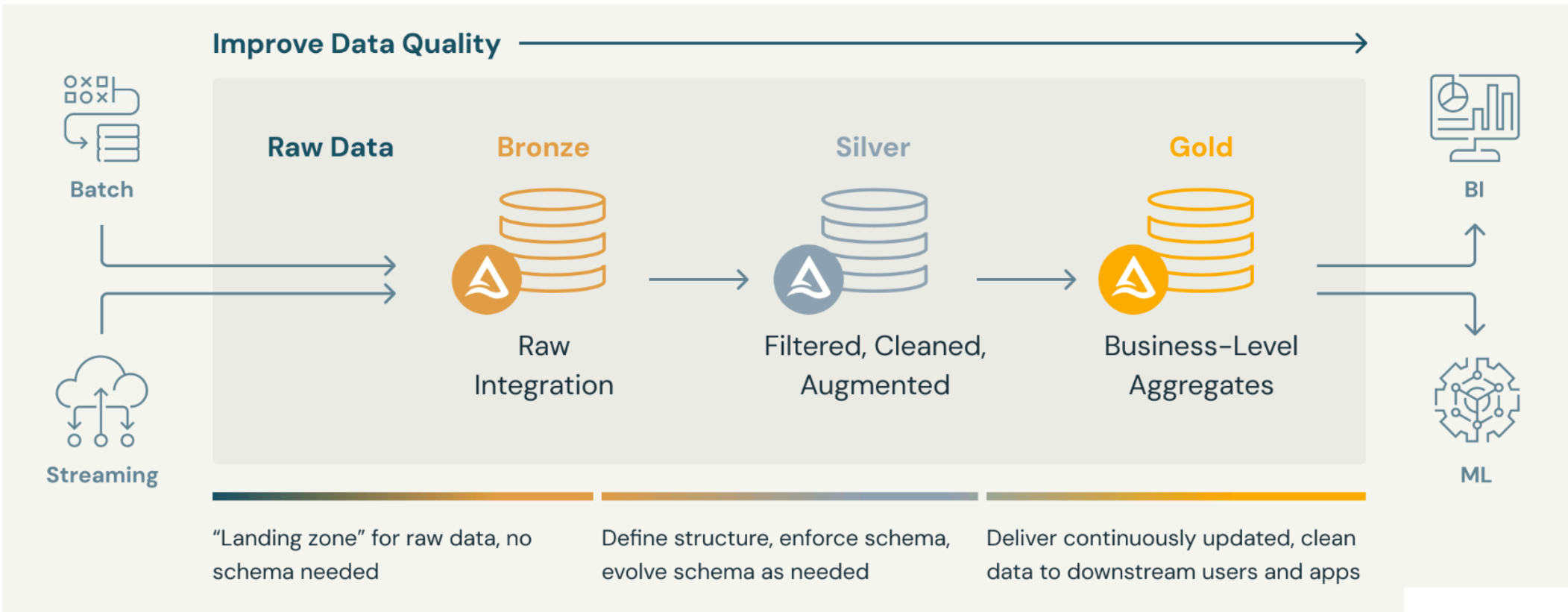
- Faster implementation, immediate business value, flexibility.

Ralph Kimball's Bottom-up approach



Medallion architecture

> is a data design pattern used to logically organize data in a lakehouse, with the goal of incrementally and progressively improving the structure and quality of data as it flows through each layer.



Medallion architecture

Important

Medallion architecture is more a guideline than a blueprint.
Learn more:

- What is a medallion architecture?
- What is wrong with medallion architecture?

Bronze layer guidelines

- capture exactly what was ingested without parsing or changes
- typically delta with date received and source / file name
- text fields with unparsed json
- other metadata
- should be append only (batch or streaming)

Silver layer guidelines

- uses Delta lake tables
- preserves grain of original data (no aggregation)
- eliminates duplicated records
- production schema enforced
- data is clean (passed data quality checks)
- corrupted data is detected and sent to quarantine
- optimized for long-term retention and ad-hoc queries

Gold layer guidelines

- uses Delta lake tables
- powers ml application, reporting and ad-hoc analytics
- allows fine grain permission
- data is optimized to reduce cost associated with queries of silver tables

Extra - Dealing with bad or invalid data

The Monday morning problem

> Friday’s revenue report said one thing. Monday’s says something else.

The Monday morning problem

> Friday’s revenue report said one thing. Monday’s says something else.

Overnight, an upstream system was upgraded. The export is subtly different.

- no job failed
- no alert fired
- the schema still matched
- the row count moved by 0.03%

The Monday morning problem

> Friday's revenue report said one thing. Monday's says something else.

Overnight, an upstream system was upgraded. The export is subtly different.

- no job failed
- no alert fired
- the schema still matched
- the row count moved by 0.03%

There is one email in your inbox. It says: *"is this number right?"*

Every job green, every number wrong

Four things arrived broken — all four of them things that really happen:

what went wrong	rows	how it happens
price is 1000× too big	40	currency stored in minor units, or a decimal shift
`customer_id` missing	40	an optional field in a new API version
price is negative	15	a refund booked as a sale
the order is duplicated	60	a retried batch that was not idempotent

Every job green, every number wrong

Four things arrived broken — all four of them things that really happen:

what went wrong	rows	how it happens
price is 1000× too big	40	currency stored in minor units, or a decimal shift
`customer_id` missing	40	an optional field in a new API version
price is negative	15	a refund booked as a sale
the order is duplicated	60	a retried batch that was not idempotent

> **Tip**
>
> Two of these push revenue **up** and two push it **down**. Which is why *“we would have noticed”* is not a plan.

One bad row

155 rows out of ~228,000 — 0.07% of the feed.

One bad row

155 rows out of ~228,000 — 0.07% of the feed.

The published revenue figure was wrong by a **double-digit percentage**. The error is **hundreds of times larger** than the defect rate.

One bad row

155 rows out of ~228,000 — 0.07% of the feed.

The published revenue figure was wrong by a **double-digit percentage**. The error is **hundreds of times larger** than the defect rate.

Because the damage lands entirely on one statistic:

summary of the same column	moved by
`count(*)`	the duplicates, and nothing else
`median(total_price)`	almost nothing
`avg(total_price)`	noticeably
`sum(total_price)`	**enormously**

One bad row

155 rows out of ~228,000 — 0.07% of the feed.

The published revenue figure was wrong by a **double-digit percentage**. The error is **hundreds of times larger** than the defect rate.

Because the damage lands entirely on one statistic:

summary of the same column	moved by
`count(*)`	the duplicates, and nothing else
`median(total_price)`	almost nothing
`avg(total_price)`	noticeably
`sum(total_price)`	**enormously**

> **Important**
>
> A 0.07% defect rate is not a 0.07% error. Aggregation **concentrates** bad values instead of diluting them — and the aggregate is what gets published.

So handle the bad rows. But how?

Four instincts. It is worth running all four rather than arguing about them.

1. **Ignore** them — the source's problem, not yours
2. **Fail** the job — refuse to publish anything until it is fixed
3. **Drop** them — filter them out and carry on
4. **Quarantine** them — divert them, with the reason attached

Same input, same three rules, four pipelines.

```
customer_id IS NOT NULL AND total_price > 0 AND total_price < 600000
```

1. Ignore

No rules. Publish what arrived.

- the job is green
- the pipeline is simple
- the number is badly wrong
- **nobody will ever find out**

1. Ignore

No rules. Publish what arrived.

- the job is green
- the pipeline is simple
- the number is badly wrong
- **nobody will ever find out**

The failure mode is not that it breaks. It is that it *doesn't*.

2. Fail the job

Refuse to load anything if any row violates a rule. This is what a `CHECK` constraint or a strict expectation does.

- the number is never wrong, because there is no number
- **228,000 good orders are held hostage by 155 bad ones**
- somebody is awake at 3am for a report that was not urgent

2. Fail the job

Refuse to load anything if any row violates a rule. This is what a `CHECK` constraint or a strict expectation does.

- the number is never wrong, because there is no number
- **228,000 good orders are held hostage by 155 bad ones**
- somebody is awake at 3am for a report that was not urgent

> **Tip**
>
> Right for a payments ledger. Wrong for a weekly sales report. The question is always *what does the consumer prefer — late, or approximate?*

3. Drop the bad rows

Deduplicate, filter out the failures, carry on. One line, and by far the most common thing people actually do.

```
gold = raw.dropDuplicates(["order_id"]).filter(RULES)
```

- the job is green
- **the number is correct**
- 155 rows are gone, and you cannot say which or why

3. Drop the bad rows

Deduplicate, filter out the failures, carry on. One line, and by far the most common thing people actually do.

```
gold = raw.dropDuplicates(["order_id"]).filter(RULES)
```

- the job is green
- **the number is correct**
- 155 rows are gone, and you cannot say which or why

> **Important**
>
> Dropping is not *wrong*. It is **unaccountable** — and those rows were real orders with real revenue, deleted permanently by a `WHERE` clause.

4. Quarantine

Same rules. The failures go to a second table with the reason attached.

```
good, bad = split_valid(raw.dropDuplicates(["order_id"]), RULES)
good.write.saveAsTable("silver_orders")
bad.write.saveAsTable("quarantine_orders")     # + _failed_rules, _quarantined_at
```

- the job is green
- the number is correct — **identical to dropping**
- and the 155 rows are still readable

The scorecard

Four pipelines, one dataset, everything measured rather than asserted.

	ignore	fail	drop	quarantine
publishes a correct number	✗	—	✓	✓
publishes *anything*	✓	✗	✓	✓
can say **what** was rejected	✗	✗	✗	✓
can **recover** the rejected rows	—	—	✗	✓
the books balance	✓	✓	✗	✓

The scorecard

Four pipelines, one dataset, everything measured rather than asserted.

	ignore	fail	drop	quarantine
publishes a correct number	✗	—	✓	✓
publishes *anything*	✓	✗	✓	✓
can say **what** was rejected	✗	✗	✗	✓
can **recover** the rejected rows	—	—	✗	✓
the books balance	✓	✓	✗	✓

Tip

The surprise is row one: **drop and quarantine publish the same number today.** They only separate on the day somebody asks a question.

Can you close the books?

> *“This is down on last year. Are we sure nothing is missing?”*

Answering means closing one equation:

```
rows received  =  rows delivered  +  rows rejected  +  duplicates removed
```

Can you close the books?

> *“This is down on last year. Are we sure nothing is missing?”*

Answering means closing one equation:

```
rows received  =  rows delivered  +  rows rejected  +  duplicates removed
```

- **drop** — 155 rows unaccounted for. Which ones? Why? The filter destroyed the evidence.
- **quarantine** — balances, itemised, and every rejected row is still readable.

Can you close the books?

> *“This is down on last year. Are we sure nothing is missing?”*

Answering means closing one equation:

```
rows received  =  rows delivered  +  rows rejected  +  duplicates removed
```

- **drop** — 155 rows unaccounted for. Which ones? Why? The filter destroyed the evidence.
- **quarantine** — balances, itemised, and every rejected row is still readable.

> **Important**
>
> **Ignore balances too**, because it rejects nothing. A closing equation proves nothing was *lost* — not that anything was *right*. Quarantine is the only column that claims both.

And they can be fixed

Those rejected rows are not garbage. They are **real orders, recorded badly**:

- the 1000× rows just need dividing
- the negative rows are refunds with the sign flipped
- the missing customer ids can be looked up in the source system

And they can be fixed

Those rejected rows are not garbage. They are **real orders, recorded badly**:

- the 1000× rows just need dividing
- the negative rows are refunds with the sign flipped
- the missing customer ids can be looked up in the source system

Re-drive them and the published number returns to **exactly** the right answer.

And they can be fixed

Those rejected rows are not garbage. They are **real orders, recorded badly**:

- the 1000× rows just need dividing
- the negative rows are refunds with the sign flipped
- the missing customer ids can be looked up in the source system

Re-drive them and the published number returns to **exactly** the right answer.

> **Important**
>
> Dropping deleted real money and left no trace. Quarantine turned Tuesday's correction into a ten-line notebook.
>
> Cost of the quarantine table: **a fraction of a percent** of the storage it protects.

What this does not cover

We have only argued that the table needs to **exist**. The discipline around it is a lecture of its own:

- the six **dimensions** of data quality, and how to turn each into a rule
- **thresholds** — must a rule hold for 100% of rows, or 99.9%?
- the **action** and the **owner** — who is called when it fires
- alerting on the **rate**, not the count
- re-drive as a **routine** rather than a heroic Tuesday
- quality metrics as a monitored table — and anomaly detection on top of them

What this does not cover

We have only argued that the table needs to **exist**. The discipline around it is a lecture of its own:

- the six **dimensions** of data quality, and how to turn each into a rule
- **thresholds** — must a rule hold for 100% of rows, or 99.9%?
- the **action** and the **owner** — who is called when it fires
- alerting on the **rate**, not the count
- re-drive as a **routine** rather than a heroic Tuesday
- quality metrics as a monitored table — and anomaly detection on top of them

> **Note**
>
> All of it in **Lecture 11 — Data quality & anomaly detection**.

Thank you!